LC 621 Task Scheduler: A Proof Journey

From a failed heap priority to frequency skeletons, alignment traps, row-based construction, and the optimal formula

Detailed English companion to the original Chinese research/debug notes. Bo Hou and ChatGPT, October 2026. This is a reconstruction of a long back-and-forth reasoning process, not a claim that each provisional hypothesis was proven. External citations are collected at the end.

0. What this document records

The important question was never merely how to memorize max(N, (f - 1)(n + 1) + k). It was how such a formula can arise from a scheduling construction, what exactly goes wrong when we freeze task positions, and what must be shown to establish global optimality. The reasoning went through multiple abandoned invariants and several reviewer errors.

The distinctive contributions in the discussion were: sorting task types by frequency U1 >= U2 >= ...; exploring the residual capacity x; recognizing alignment as a thought attractor; constructing rows of length n+1; aligning common tasks in adjacent rows; preserving top-k frequency leaders via tie-breaking rather than relying on a permutation metaphor; and refusing to equate a few verified examples with a general theorem.

1. Problem statement and notation

Each execution takes one unit of time. The same task type must have at least n intervening time slots (possibly idle); tasks may be reordered. Minimize the total number of slots. This is LeetCode 621 [1].

```
N = sum of all task frequencies
U1 >= U2 >= ... >= Um = sorted frequencies
f = U1 = maximum frequency
k = number of types occurring f times
L = n + 1 = minimum position difference for repeated tasks
```

A lower bound rules out shorter schedules; a feasible construction provides an upper bound; optimality requires those bounds to coincide. Much of the conversation stalled when these three logical claims were conflated.

2. Round 1: One min-heap, one wrong priority

The original proposal attached each task type a next-available time and a remaining count, then ordered a min-heap by (availability, negative remaining count). Its benefit was event-driven simulation: time could jump to the earliest release event instead of scanning a gigantic time range.

The flaw is subtle: the earliest release time is a feasibility constraint, not an optimization priority among tasks that are already available. If two types can both execute now, taking the one released earlier can postpone a high-frequency type and introduce idle at the tail.

```
tasks = A A A B C; n = 1
Earlier-availability priority may choose: A B C A _ A  (6 slots)
A different feasible order is:    A B A C A    (5 slots)
```

This counterexample establishes failure of that specific heap key, not failure of the heap data structure itself. The next candidate was: among currently available types, favor larger remaining counts, while separately tracking

cooldown releases.

3. Round 2: The U1 skeleton and residual capacity

Bo began with the most frequent task type, U1. Its copies impose f-1 cooldown gaps. The natural next move is to insert U2, U3, and so on, by descending frequency. This is much more specific than saying merely “greedy packing”: it fixes a prefix of the timetable and tracks capacity that remains after assigning particular types.

At one stage the quantity x = U1 - (U2 + ... + Ui) was introduced for a restricted packing abstraction. When a new type Ui+1 does not fit, the remaining Ui+1-x executions appear to form a smaller problem. That resemblance to the original problem was tempting but unproven.

The user identified the problem himself: the already scheduled prefix fixes previous execution times of partially used types. The residual task multiset no longer captures all constraints. Therefore solving the remainder as an independent instance does not automatically preserve global optimality.

4. The failed-invariant search log

Shortest finish for every type

Minimizing the completion time of each type separately cannot be assumed to minimize the joint makespan: the types share a single CPU.

Frozen optimal prefix

Once task positions are frozen, future packing has inherited cooldown constraints. A prefix that looked optimal by itself need not appear in any globally optimal schedule.

Residual-subproblem equivalence

Subtracting residual capacity x and sorting the remainder does not erase history. The smaller-looking subproblem need not be equivalent to the original one.

Optimal segments compose

Optimizing two segments separately and concatenating them fails to account for cooldown requirements across the junction.

Remaining task count equals time increment

A count of remaining executions alone cannot determine the required extension if the frozen prefix induces additional waiting.

Enough empty slots implies feasibility

Total vacant capacity is not a complete feasibility condition: repetitions must be separated. Some placement discipline or an existence proof is necessary.

Local slot filling keeps an optimal invariant

Repeated attempts to prove a stepwise optimal-prefix invariant returned to the same unresolved alignment condition.

5. The alignment thought attractor

The user named an important methodological failure: alignment was a “thought attractor.” Each repair to a frozen skeleton led back to which task should occupy which exact position, how to append Ui+1, and how to realign after partial packing. It was possible to spend indefinitely refining examples while never establishing a usable invariant.

The lesson was not that packing constructions are invalid. It was that an abstraction should be changed when the desired invariant repeatedly requires a complete solution to the original problem. Freezing U1 is a construction choice, not an intrinsic property of all optimal timetables.

6. A concrete 13-slot versus 11-slot lesson

For n=2 and frequencies A=3, B=C=D=E=2, a frozen-pattern construction may yield a legal schedule of 13 slots:

```
position:  1  2  3  4  5  6  7  8  9 10 11 12 13
schedule:  A  B  C  D  A  B  C  D  A  E  _  _  E
```

This uses all eleven tasks and respects cooldown; however a legal 11-slot schedule exists:

```
A B C D E | A B C D E | A
```

The point of the dispute was not to reverse-engineer the correct arrangement. A valid length-11 schedule proves an upper bound for that instance but does not, by itself, explain an algorithm generating optimal schedules generally. Multiple reviews of this example also confused obvious typographical mistakes with conceptual flaws, which substantially distracted from the actual proof problem.

7. The new construction: rows of exactly n+1 slots

The conceptual pivot was to stop pre-allocating every occurrence of U1. Instead divide the schedule into rows of L=n+1 consecutive positions. In each round, select up to L distinct task types with the largest remaining frequencies. Execute each selected type once, decrease its count, and repeat.

```
Row length L = n+1
At each iteration:
  choose up to L distinct types with highest positive frequencies
  assign these types to row columns
  place repeated types shared with previous row in the same column
  decrement selected frequencies
  use idle only if a round genuinely requires it
```

This is the core construction in the discussion. It must not be confused with the final closed-form calculation, which does not explicitly schedule tasks.

8. Why the alignment condition becomes local

If type A appears in adjacent rows, giving it the same column produces a position difference of L=n+1. If A skips at least one whole row, then even the last column of its first row and first column of the later row are separated by at least L+1 positions. Thus nonadjacent repetitions do not require exact column alignment.

For two given adjacent sets of distinct task types, common types can occupy matching columns, while unique types fill remaining columns. This establishes pairwise compatibility. Extending the construction over every

round additionally requires a consistent online column assignment, not simply an appeal to pairwise feasibility.

9. Why frequency selection and placement are different claims

Frequency selection determines which types are present in a row. Column assignment makes the resulting rows satisfy cooldown. The discussion initially conflated these two tasks, repeatedly treating every newly discovered alignment concern as a counterexample to the whole greedy algorithm.

The user correctly insisted that an adjacent-row alignment rule addresses one specific obligation. Another obligation remains: to prove that the selection policy does not force avoidable idle, or that any idle it introduces is necessary under the global frequency constraints.

10. Case k < n+1: preserve the initial leaders

When initially k types tie for maximum frequency and k<L, each row has enough capacity to execute all k of them. If a lower-frequency type catches up, tie-breaking can continue to select the original k leaders. Their counts then decrease in lockstep and they can remain aligned across adjacent rows.

The user briefly explored “implicit replacement” or permutation of top-k roles after frequency ties, but abandoned it for this cleaner statement. In this case, protecting the original k leaders directly gives a simpler invariant. This should be described as a case-specific invariant, not a universal requirement in every k regime.

11. Case k >= n+1: the row capacity is saturated

When there are at least L top-frequency task types, a single row cannot contain all of them. The same selection rule nonetheless continues: choose L distinct maximum-remaining types, decrement, rerank, and assign columns consistent with the predecessor row. Leaders rotate naturally across rows. No new scheduling rule is needed.

```
n = 2, L = 3; A=B=C=D=E=3
Row 1: A B C
Row 2: D E C
Row 3: D E A
Row 4: B C A
Row 5: B D E
```

This particular construction is legal: repeated tasks in adjacent rows stay in the same column; all 15 slots are occupied. It demonstrates the k>L boundary condition but is not itself a universal inductive proof for arbitrary initial frequency vectors.

12. Two lower bounds, and the missing sufficiency theorem

First, total work requires at least N slots. Second, f repetitions of every maximally frequent type force f-1 separation spans of length at least n+1, plus a final block for k equally frequent types. The familiar bound is:

```
T >= N
T >= (f-1)*(n+1) + k
Optimal T = max(N, (f-1)*(n+1) + k)  [standard result: 2,3]
```

The nontrivial direction is the upper bound: constructing a schedule that attains the larger lower bound. The row idea is a route toward such an existence argument. Merely saying “there are enough tasks, so there will be no

idle” is not a proof. The Chinese research log deliberately records this gap instead of laundering it into a theorem.

13. Implementation: count-first formula

```
int leastInterval(vector<char>& tasks, int n) {
    int cnt[26] = {};
    for (char c : tasks) ++cnt[c - 'A'];
    int f = *max_element(cnt, cnt + 26);
    int k = 0;
    for (int x : cnt) if (x == f) ++k;
    return max((int)tasks.size(), (f - 1)*(n+1) + k);
}
```

With 26 fixed task types, frequency counting is O(N) time and O(1) additional space. A simulation with heaps is useful for understanding the schedule and producing an actual arrangement, but the task asks only for the minimum length.

14. Review and collaboration lessons

A mathematical review must either produce a concrete counterexample or identify a precise failed implication. Repeating “not yet proven optimal” after the author has already acknowledged the gap contributes little. The assistant repeatedly mixed up task counts, idle slots, example strings, and case premises, and sometimes used the final formula to justify the very construction under investigation.

The more useful record is the chain of hypotheses, failed invariants, and corrected assumptions: what Bo proposed, what was and was not established, and which review statements were simply wrong. A clean final formula should not erase that reasoning trail.

15. References and provenance

[1] LeetCode, Problem 621: Task Scheduler. https://leetcode.com/problems/task-scheduler/

[2] NeetCode, Task Scheduler explanation. https://neetcode.io/solutions/task-scheduler

[3] Doocs LeetCode Wiki, Problem 621. https://leetcode.doocs.org/en/lc/621/

All discussion-derived claims (the notation U1 and x, the alignment attractor, the row construction, tie-breaking proposal, review misjudgments, and exploratory invariants) originate from the user–assistant discussion rather than external sources. This manuscript is an English companion narrative, not a literal transcript or independently peer-reviewed publication.